Plataforma de Torneos Beyblade: Requisitos y Esquema de Base de Datos

Versión 0.2 · Documento de partida para desarrollo con Claude Code · Stack: Next.js + TypeScript (Vercel), Supabase (Postgres, Auth, RLS, Realtime), GitHub

1. Alcance del MVP

Plataforma web para crear, configurar y arbitrar torneos de Beyblade X con la capacidad de edición de torneos de Challonge adaptada a Beyblade.

Incluye en el MVP

- Formatos: eliminación individual, doble eliminación, todos contra todos (round robin) y sistema suizo.
- Torneos multietapa: varias etapas encadenadas (por ejemplo Swiss, luego grupos, luego eliminación directa).
- Fase de grupos generada a partir de los clasificados de la etapa anterior, con bracket posterior que puede iniciar desde 16vos (32 jugadores) con byes si no se llena.
- Arbitraje por batalla con los cuatro tipos de finish de Beyblade X, tarjetas y bitácora de cambios.
- Vista pública en tiempo real sin necesidad de cuenta.

Fuera del MVP (versiones posteriores): rol blader con autoinscripción en torneos, registro de decks y catálogo de piezas, pantalla de estadio para proyector, ranking ELO por temporada, pagos en línea, transmisión en vivo y otros formatos (free for all, liga).

2. Roles

- **Admin:** crea y asigna las cuentas de organizador y árbitro; es el único que cambia roles (incluido otro admin).
- **Organizador:** crea y edita sus torneos, agrega participantes, asigna árbitros, inicia etapas y corrige resultados.
- **Árbitro:** registra batallas y tarjetas solo en los matches que tiene asignados.
- **Público:** consulta torneos públicos, brackets y resultados sin iniciar sesión.

Acceso: inicio de sesión con Supabase Auth (correo y contraseña). El registro público queda cerrado: el admin invita a organizadores y árbitros. El rol **blader** (autoinscripción en torneos y perfil propio) se agrega en una etapa posterior sin cambiar el modelo, porque los participantes ya pueden existir sin cuenta.

3. Requisitos funcionales

3.1 Creación y edición de torneos

- **RF-01** Crear torneo con nombre, descripción, fecha, cupo máximo, visibilidad (público o privado), tarifa y generación de juego (Beyblade X por defecto).
- **RF-02** Configurar los datos del evento: sede, dirección, país y fecha.
- **RF-03** Configurar puntos por tipo de finish por torneo (valores por defecto: Spin 1, Burst 2, Over 2, Xtreme 3).
- **RF-04** Definir una o más etapas ordenadas. Cada etapa tiene tipo, match type (1on1 o 3on3), puntos para ganar, mejor de N sets (best_of: 3 por defecto en eliminación, 1 en grupos y suizo) y parámetros propios del formato.
- **RF-05** Vista previa del bracket o del calendario de la etapa antes de iniciar.
- **RF-06** Editar torneo y etapas mientras estén en borrador. Una vez iniciada la etapa, solo se permiten cambios controlados (descalificar, reemplazar, corregir resultado).
- **RF-07** Duplicar un torneo como plantilla.

3.2 Participantes

- **RF-08** El organizador agrega a los participantes manualmente, uno a uno o en bloque, aunque no tengan cuenta (walk-in). La autoinscripción de bladers queda para una etapa posterior.
- **RF-09** Check-in opcional antes del inicio.
- **RF-10** Siembra (seeding): manual, aleatoria o por ranking. Reordenar y resembrar antes de iniciar.
- **RF-11** Check-in masivo (todos a la vez) además del individual, y retiro de participantes antes del inicio.
- **RF-12** Retiro o descalificación de un participante, con reflujo automático en los matches pendientes.

3.3 Etapas y formatos

- **RF-13** Eliminación individual con byes, partido por el tercer lugar opcional.
- **RF-14** Doble eliminación con bracket de perdedores y final con reinicio opcional (grand final reset).
- **RF-15** Todos contra todos, con una o dos vueltas y tabla de posiciones; los empates son válidos (3 puntos por victoria y 1 por empate).
- **RF-16** Suizo: número de rondas configurable, emparejamiento por récord similar sin repetir rivales, manejo de bye.
- **RF-17** Fase de grupos: tamaño de grupo y cantidad de clasificados configurables por etapa y ajustables por grupo.
- **RF-18** Transición entre etapas: al cerrar una etapa, el sistema calcula los clasificados según la tabla y los desempates de la etapa (por ejemplo, los 4 o 3 primeros de cada grupo) y genera la siguiente (grupos o bracket desde 16vos, 8vos, etc.) respetando la siembra.
- **RF-19** Desempates configurables por etapa: el organizador ordena los criterios (puntos, diferencia de puntos, puntos a favor, enfrentamiento directo, Buchholz y siembra). Por defecto: puntos, diferencia de puntos, puntos a favor y siembra inicial en grupos; Buchholz en suizo.

3.4 Arbitraje

- **RF-20** Panel de árbitro optimizado para móvil: lista de matches asignados, inicio de match y registro batalla por batalla.
- **RF-21** Cada batalla registra: quién anotó, tipo de finish, puntos y número de batalla (o empate). Incluye deshacer.
- **RF-22** El marcador del match se recalcula automáticamente. Al alcanzar los puntos requeridos, el match se cierra y el ganador avanza.
- **RF-23** En 3on3 sin ganador tras tres rondas: reordenar los beys y jugar la batalla decisiva.
- **RF-24** Tarjetas amarilla y roja con motivo; la amarilla por no-contacto anula la batalla y obliga a repetirla. Cada 3 faltas de un participante en el match dan 1 punto al rival (ambos valores configurables por torneo).
- **RF-25** Corrección o reapertura de resultados ya cerrados solo por organizador o admin, con reflujo del bracket y registro en la bitácora de auditoría.
- **RF-26** Bey Check: marcar al participante como inspeccionado y anotar sustituciones.

3.5 Vista pública y tiempo real

- **RF-27** Enlace público por torneo con bracket, grupos, tabla de posiciones y resultados.
- **RF-28** Actualización en vivo (Supabase Realtime) cuando se registra una batalla.
- **RF-29** Exportar resultados (CSV) y compartir el enlace.

4. Requisitos no funcionales

- **RNF-01** El motor de torneos se implementa como funciones puras en TypeScript, sin dependencia de la base de datos, con cobertura de pruebas unitarias alta (es el componente más crítico).
- **RNF-02** Seguridad con Row Level Security en todas las tablas; ningún dato sensible accesible desde el cliente sin política.
- **RNF-03** Concurrencia: versión optimista en cada match y registro de batallas en una función SQL transaccional (record_battle), para que dos árbitros, o un deshacer simultáneo, no corrompan el marcador.
- **RNF-04** Diseño responsive, prioridad móvil para árbitros y espectadores.
- **RNF-05** Todas las migraciones SQL versionadas en el repositorio (Supabase CLI) y CI en GitHub Actions (lint, tests, build).
- **RNF-06** Interfaz y textos en español; estructura lista para internacionalizar.
- **RNF-07** Autenticación solo con Supabase Auth: sin JWT propio ni contraseñas de admin en variables de entorno; el primer admin se asigna con un SQL de una sola vez.
- **RNF-08** Sin estado en memoria del proceso ni modo sin base de datos: la app es sin estado (Vercel) y toda la información vive en Postgres. JSON solo para la configuración de cada etapa, nunca duplicando datos relacionales.
- **RNF-09** Validar toda entrada con Zod en cada ruta del servidor y devolver errores en un formato uniforme (código, mensaje y campo).
- **RNF-10** Avanzar el bracket en una sola transacción (función RPC), para que no queden matches a medias si algo falla.
- **RNF-11** Reconexión automática de Realtime y reintentos seguros (idempotentes) en el panel de árbitro ante fallos de red.
- **RNF-12** Registro de errores (por ejemplo Sentry), endpoint de salud y pruebas de regresión de grupos, suizo y ciclo de vida de los matches.

5. Reglas de negocio de Beyblade X

- Puntos por defecto: Spin Finish 1, Burst Finish 2, Over Finish 2, Xtreme Finish 3. Cada 3 faltas dan 1 punto al rival y un empate no da puntos.
- Puntos para ganar (estándar WBO): 4 en la primera etapa y 7 en la etapa final. Configurable por etapa, con opción de mejor de N sets.
- Grupos y round robin: 3 puntos por victoria y 1 por empate.
- Si un bey cae en un pocket sin contacto previo: tarjeta amarilla, sin puntos y repetición.
- Cada etapa define su propio match type y puntaje, de modo que una etapa suiza puede jugarse a 4 puntos y la final a 7.

6. Modelo de datos (resumen)

- **profiles:** cuentas y su rol (admin, organizador, árbitro).
- **tournaments:** configuración general y datos del evento (sede, dirección, país).
- **tournament_finish_points:** puntos por tipo de finish en cada torneo.
- **tournament_referees:** árbitros asignados a un torneo.
- **participants:** inscritos con o sin cuenta, siembra, check-in, inspección y estado.
- **stages:** etapas ordenadas con su tipo, puntaje, best_of y configuración en JSON.
- **groups y group_participants:** grupos generados dentro de una etapa.
- **rounds:** rondas de cada etapa o grupo (ganadores, perdedores, final, grupo, suizo).
- **matches:** enfrentamientos con empates, byes y enlaces al match siguiente del ganador y del perdedor.
- **battles:** cada batalla individual dentro de un match, o un empate.
- **penalties:** tarjetas y puntos otorgados al rival.
- **audit_log:** bitácora de correcciones.

7. Esquema SQL para Supabase (migración inicial)

```
-- Tipos enumerados
create type user_role as enum ('admin','organizer','referee'); -- 'blader' se
agrega despues: alter type user_role add value 'blader';
create type tournament_status as enum
('draft','registration','in_progress','completed','cancelled');
create type stage_type as enum
('single_elimination','double_elimination','round_robin','swiss','group_stage
```

```
');
create type stage_status as enum ('pending','in_progress','completed');
create type match_status as enum ('pending','in_progress','completed');
create type match_type as enum ('1on1','3on3');
create type finish_type as enum ('spin','over','burst','xtreme');
create type card_type as enum ('yellow','red');
create type participant_status as enum
('registered','active','eliminated','disqualified','withdrawn');

-- Perfiles (extiende auth.users). Las cuentas las invita el admin.
create table profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  display_name text not null,
  role user_role not null default 'referee', -- minimo privilegio; solo el
admin lo cambia
  created_at timestamptz not null default now()
);

-- Torneos
create table tournaments (
  id uuid primary key default gen_random_uuid(),
  slug text unique not null,
  name text not null,
  description text,
  organizer_id uuid not null references profiles(id),
  status tournament_status not null default 'draft',
  is_public boolean not null default true,
  max_participants int,
  check_in_required boolean not null default false,
  entry_fee numeric(10,2) default 0, foul_points int not null default 1 check
(foul_points >= 0), fouls_per_point int not null default 3 check
(fouls_per_point > 0),
  venue_name text,
  venue_address text,
  country text default 'PA',
  starts_at timestamptz,
  created_at timestamptz not null default now()
);

create table tournament_finish_points (
  tournament_id uuid references tournaments(id) on delete cascade,
  finish finish_type,
  points int not null check (points >= 0),
```

```
  primary key (tournament_id, finish)
);

-- Puntos por defecto al crear el torneo (spin 1, over 2, burst 2, xtreme 3)
create function seed_finish_points() returns trigger language plpgsql
security definer set search_path = public as $$
begin
  insert into tournament_finish_points (tournament_id, finish, points)
  values (new.id,'spin',1),(new.id,'over',2),(new.id,'burst',2),
(new.id,'xtreme',3);
  return new;
end $$;
create trigger tournaments_seed_points after insert on tournaments
  for each row execute function seed_finish_points();

create table tournament_referees (
  tournament_id uuid references tournaments(id) on delete cascade,
  profile_id uuid references profiles(id) on delete cascade,
  primary key (tournament_id, profile_id)
);

-- Participantes (profile_id nulo = participante sin cuenta; el rol blader
llegara despues)
create table participants (
  id uuid primary key default gen_random_uuid(),
  tournament_id uuid not null references tournaments(id) on delete cascade,
  profile_id uuid references profiles(id),
  display_name text not null,
  seed int,
  checked_in boolean not null default false,
  inspected boolean not null default false,
  status participant_status not null default 'registered',
  unique (tournament_id, seed)
);

-- Etapas, grupos y rondas
create table stages (
  id uuid primary key default gen_random_uuid(),
  tournament_id uuid not null references tournaments(id) on delete cascade,
  position int not null,
  name text not null,
  type stage_type not null,
  status stage_status not null default 'pending',
```

```
  match_type match_type not null default '3on3',
  points_to_win int not null default 4 check (points_to_win > 0),
  best_of int not null default 1 check (best_of in (1,3,5,7)),
  config jsonb not null default '{}'::jsonb,
  unique (tournament_id, position)
);
-- config ejemplo: {"tiebreakers":
["points","point_diff","points_for","seed"],"advance_count":2,"group_size":4,
"swiss_rounds":5,"grand_final_reset":true,"third_place_match":true,"start_rou
nd":"round_of_32"}

create table groups (
  id uuid primary key default gen_random_uuid(),
  stage_id uuid not null references stages(id) on delete cascade,
  name text not null,
  position int not null
);

create table group_participants (
  group_id uuid references groups(id) on delete cascade,
  participant_id uuid references participants(id) on delete cascade,
  primary key (group_id, participant_id)
);

create table rounds (
  id uuid primary key default gen_random_uuid(),
  stage_id uuid not null references stages(id) on delete cascade,
  group_id uuid references groups(id) on delete cascade,
  number int not null,
  bracket text not null check (bracket in
('winners','losers','grand_final','group','swiss')),
  name text
);

-- Matches y batallas. score1/score2 = puntos del set en curso; sets1/sets2 =
sets ganados.
create table matches (
  id uuid primary key default gen_random_uuid(),
  tournament_id uuid not null references tournaments(id) on delete cascade,
  stage_id uuid not null references stages(id) on delete cascade,
  round_id uuid not null references rounds(id) on delete cascade,
  group_id uuid references groups(id),
  position int not null,
```

```
  participant1_id uuid references participants(id),
  participant2_id uuid references participants(id),
  winner_id uuid references participants(id),
  is_draw boolean not null default false,
  is_bye boolean not null default false,
  score1 int not null default 0,
  score2 int not null default 0,
  sets1 int not null default 0,
  sets2 int not null default 0,
  status match_status not null default 'pending',
  referee_id uuid references profiles(id),
  next_match_winner_id uuid references matches(id),
  next_match_loser_id uuid references matches(id),
  version int not null default 1,
  started_at timestamptz,
  finished_at timestamptz,
  check (not is_draw or winner_id is null)
);

-- Batalla; scorer_id y finish nulos = empate (0 puntos)
create table battles (
  id uuid primary key default gen_random_uuid(),
  match_id uuid not null references matches(id) on delete cascade,
  battle_number int not null,
  set_number int not null default 1,
  scorer_id uuid references participants(id),
  finish finish_type,
  points int not null default 0,
  voided boolean not null default false,
  recorded_by uuid references profiles(id),
  created_at timestamptz not null default now(),
  unique (match_id, battle_number),
  check ((scorer_id is null) = (finish is null))
);

-- Tarjetas y faltas; una falta puede dar puntos al rival
create table penalties (
  id uuid primary key default gen_random_uuid(),
  match_id uuid not null references matches(id) on delete cascade,
  battle_id uuid references battles(id),
  participant_id uuid not null references participants(id),
  card card_type not null,
  is_foul boolean not null default true, points_awarded int not null default
```

```
0,
  awarded_to uuid references participants(id),
  reason text,
  issued_by uuid references profiles(id),
  created_at timestamptz not null default now()
);

create table audit_log (
  id bigint generated always as identity primary key,
  table_name text not null,
  record_id uuid,
  action text not null,
  old_data jsonb,
  new_data jsonb,
  actor uuid,
  created_at timestamptz not null default now()
);

-- Indices clave
create index on participants (tournament_id);
create index on matches (stage_id, round_id);
create index on matches (referee_id) where status <> 'completed';
create index on battles (match_id);
```

8. Seguridad (RLS, política base)

```
alter table profiles enable row level security;
alter table tournaments enable row level security;
alter table tournament_finish_points enable row level security;
alter table tournament_referees enable row level security;
alter table participants enable row level security;
alter table stages enable row level security;
alter table groups enable row level security;
alter table group_participants enable row level security;
alter table rounds enable row level security;
alter table matches enable row level security;
alter table battles enable row level security;
alter table penalties enable row level security;
alter table audit_log enable row level security;

-- Funciones auxiliares
```

```
create function is_admin() returns boolean language sql stable security
definer set search_path = public as $$
  select exists (select 1 from profiles where id = auth.uid() and role =
'admin');
$$;

create function is_organizer() returns boolean language sql stable security
definer set search_path = public as $$
  select exists (select 1 from profiles where id = auth.uid() and role in
('admin','organizer'));
$$;

create function manages_tournament(t uuid) returns boolean language sql
stable security definer set search_path = public as $$
  select is_admin() or exists (select 1 from tournaments where id = t and
organizer_id = auth.uid());
$$;

create function referees_match(m uuid) returns boolean language sql stable
security definer set search_path = public as $$
  select exists (select 1 from matches where id = m and referee_id =
auth.uid());
$$;

create function can_read_tournament(t uuid) returns boolean language sql
stable security definer set search_path = public as $$
  select exists (
    select 1 from tournaments where id = t and (is_public or
manages_tournament(t)
      or exists (select 1 from tournament_referees r where r.tournament_id =
t and r.profile_id = auth.uid()))
  );
$$;

-- Perfil automatico al invitar un usuario (desactiva el registro publico en
Supabase Auth)
create function handle_new_user() returns trigger language plpgsql security
definer set search_path = public as $$
begin
  insert into profiles (id, display_name)
  values (new.id, coalesce(new.raw_user_meta_data->>'display_name',
new.email));
  return new;
```

```
end $$;
create trigger on_auth_user_created after insert on auth.users
  for each row execute function handle_new_user();

-- Perfiles: cada quien ve el suyo; solo el admin ve todos y cambia roles
create policy profiles_read on profiles for select using (id = auth.uid() or
is_admin());
create policy profiles_admin_update on profiles for update using (is_admin())
with check (is_admin());

-- Torneos
create policy tournaments_read on tournaments for select using
(can_read_tournament(id));
create policy tournaments_insert on tournaments for insert with check
(is_organizer() and organizer_id = auth.uid());
create policy tournaments_update on tournaments for update using
(manages_tournament(id)) with check (manages_tournament(id));
create policy tournaments_delete on tournaments for delete using
(manages_tournament(id));

-- Tablas con tournament_id: lectura segun visibilidad, escritura del
organizador o admin
create policy points_read on tournament_finish_points for select using
(can_read_tournament(tournament_id));
create policy points_manage on tournament_finish_points for all using
(manages_tournament(tournament_id)) with check
(manages_tournament(tournament_id));
create policy referees_read on tournament_referees for select using
(can_read_tournament(tournament_id));
create policy referees_manage on tournament_referees for all using
(manages_tournament(tournament_id)) with check
(manages_tournament(tournament_id));
create policy participants_read on participants for select using
(can_read_tournament(tournament_id));
create policy participants_manage on participants for all using
(manages_tournament(tournament_id)) with check
(manages_tournament(tournament_id));
create policy stages_read on stages for select using
(can_read_tournament(tournament_id));
create policy stages_manage on stages for all using
(manages_tournament(tournament_id)) with check
(manages_tournament(tournament_id));
create policy matches_read on matches for select using
```

```
(can_read_tournament(tournament_id));
create policy matches_manage on matches for all using
(manages_tournament(tournament_id)) with check
(manages_tournament(tournament_id));

-- Tablas que cuelgan de etapas o matches
create policy groups_read on groups for select using (exists (select 1 from
stages s where s.id = stage_id and can_read_tournament(s.tournament_id)));
create policy groups_manage on groups for all using (exists (select 1 from
stages s where s.id = stage_id and manages_tournament(s.tournament_id)));
create policy group_participants_read on group_participants for select using
(exists (select 1 from groups g join stages s on s.id = g.stage_id where g.id
= group_id and can_read_tournament(s.tournament_id)));
create policy group_participants_manage on group_participants for all using
(exists (select 1 from groups g join stages s on s.id = g.stage_id where g.id
= group_id and manages_tournament(s.tournament_id)));
create policy rounds_read on rounds for select using (exists (select 1 from
stages s where s.id = stage_id and can_read_tournament(s.tournament_id)));
create policy rounds_manage on rounds for all using (exists (select 1 from
stages s where s.id = stage_id and manages_tournament(s.tournament_id)));
create policy battles_read on battles for select using (exists (select 1 from
matches m where m.id = match_id and can_read_tournament(m.tournament_id)));
create policy battles_manage on battles for all using (exists (select 1 from
matches m where m.id = match_id and manages_tournament(m.tournament_id)));
create policy penalties_read on penalties for select using (exists (select 1
from matches m where m.id = match_id and
can_read_tournament(m.tournament_id)));
create policy penalties_manage on penalties for all using (exists (select 1
from matches m where m.id = match_id and
manages_tournament(m.tournament_id)));
create policy penalties_referee_insert on penalties for insert with check
(referees_match(match_id));
create policy audit_admin_read on audit_log for select using (is_admin());

-- Registro transaccional de una batalla (los arbitros NO insertan en battles
directamente)
create function record_battle(p_match uuid, p_scorer uuid, p_finish
finish_type, p_expected_version int)
returns matches language plpgsql security definer set search_path = public as
$$
declare
  m matches;
  s stages;
```

```
  pts int;
  need int;
  scorer_is_p1 boolean;
begin
  select * into m from matches where id = p_match for update;
  if not found then raise exception 'match_not_found'; end if;
  if not (referees_match(p_match) or manages_tournament(m.tournament_id))
then raise exception 'forbidden'; end if;
  if m.status = 'completed' then raise exception 'match_closed'; end if;
  if m.version <> p_expected_version then raise exception 'version_conflict';
end if;
  if m.participant1_id is null or m.participant2_id is null then raise
exception 'match_not_ready'; end if;
  if p_scorer not in (m.participant1_id, m.participant2_id) then raise
exception 'invalid_scorer'; end if;

  select * into s from stages where id = m.stage_id;
  select points into pts from tournament_finish_points where tournament_id =
m.tournament_id and finish = p_finish;
  scorer_is_p1 := (p_scorer = m.participant1_id);

  insert into battles (match_id, battle_number, set_number, scorer_id,
finish, points, recorded_by)
  values (p_match, (select coalesce(max(battle_number), 0) + 1 from battles
where match_id = p_match),
          m.sets1 + m.sets2 + 1, p_scorer, p_finish, pts, auth.uid());

  if scorer_is_p1 then m.score1 := m.score1 + pts; else m.score2 := m.score2
+ pts; end if;
  m.status := 'in_progress';

  if greatest(m.score1, m.score2) >= s.points_to_win then
    if scorer_is_p1 then m.sets1 := m.sets1 + 1; else m.sets2 := m.sets2 + 1;
end if;
    need := s.best_of / 2 + 1;
    if m.sets1 >= need or m.sets2 >= need then
      m.status := 'completed';
      m.winner_id := case when m.sets1 >= need then m.participant1_id else
m.participant2_id end;
      m.finished_at := now();
    else
      m.score1 := 0;
      m.score2 := 0;
```

```
    end if;
  end if;

  update matches set score1 = m.score1, score2 = m.score2, sets1 = m.sets1,
sets2 = m.sets2,
    status = m.status, winner_id = m.winner_id, finished_at = m.finished_at,
    started_at = coalesce(started_at, now()), version = version + 1
  where id = p_match returning * into m;
  return m;
end $$;
```

Registro de faltas: la función record_penalty guarda la tarjeta y cuenta las faltas del participante en el match; cada vez que llega a un múltiplo de fouls_per_point (3 por defecto) suma al rival foul_points (1 por defecto), con la misma lógica de sets que record_battle. Una tarjeta que no es falta (is_foul falso) solo se registra.

```
create function record_penalty(p_match uuid, p_offender uuid, p_card
card_type, p_is_foul boolean, p_reason text, p_expected_version int)
returns matches language plpgsql security definer set search_path = public as
$$
declare
  m matches;
  s stages;
  fp int;
  per int;
  nfouls int;
  pts int := 0;
  rival uuid;
  rival_is_p1 boolean;
  need int;
begin
  select * into m from matches where id = p_match for update;
  if not found then raise exception 'match_not_found'; end if;
  if not (referees_match(p_match) or manages_tournament(m.tournament_id))
then raise exception 'forbidden'; end if;
  if m.status = 'completed' then raise exception 'match_closed'; end if;
  if m.version <> p_expected_version then raise exception 'version_conflict';
end if;
  if p_offender not in (m.participant1_id, m.participant2_id) then raise
exception 'invalid_participant'; end if;

  select * into s from stages where id = m.stage_id;
  select foul_points, fouls_per_point into fp, per from tournaments where id
```

```
= m.tournament_id;

  -- cada 'per' faltas del mismo participante en el match, el rival suma 'fp'
puntos
  if p_is_foul then
    select count(*) + 1 into nfouls from penalties
      where match_id = p_match and participant_id = p_offender and is_foul;
    if nfouls % per = 0 then pts := fp; end if;
  end if;

  rival := case when p_offender = m.participant1_id then m.participant2_id
else m.participant1_id end;
  rival_is_p1 := (rival = m.participant1_id);

  insert into penalties (match_id, participant_id, card, is_foul,
points_awarded, awarded_to, reason, issued_by)
  values (p_match, p_offender, p_card, p_is_foul, pts, case when pts > 0 then
rival end, p_reason, auth.uid());

  m.status := 'in_progress';
  if pts > 0 then
    if rival_is_p1 then m.score1 := m.score1 + pts; else m.score2 := m.score2
+ pts; end if;
    if greatest(m.score1, m.score2) >= s.points_to_win then
      if rival_is_p1 then m.sets1 := m.sets1 + 1; else m.sets2 := m.sets2 +
1; end if;
      need := s.best_of / 2 + 1;
      if m.sets1 >= need or m.sets2 >= need then
        m.status := 'completed';
        m.winner_id := case when m.sets1 >= need then m.participant1_id else
m.participant2_id end;
        m.finished_at := now();
      else
        m.score1 := 0;
        m.score2 := 0;
      end if;
    end if;
  end if;

  update matches set score1 = m.score1, score2 = m.score2, sets1 = m.sets1,
sets2 = m.sets2,
    status = m.status, winner_id = m.winner_id, finished_at = m.finished_at,
```

```
    started_at = coalesce(started_at, now()), version = version + 1
  where id = p_match returning * into m;
  return m;
end $$;
```

Las batallas se registran solo con la función record_battle, que bloquea el match, valida la versión y actualiza el marcador en una sola transacción; el organizador y el admin pueden corregirlas directamente. El avance del ganador al match siguiente lo hace una segunda función (o el motor TypeScript dentro de la misma transacción). Antes de producción: desactiva el registro público en Supabase Auth y asigna el primer admin una sola vez con `update profiles set role = 'admin' where id = '<uuid del usuario>'`.

9. Motor de torneos (funciones puras)

- `generateSingleElimination(participants, options)` y `generateDoubleElimination(participants, options)`: calculan rondas, byes y enlaces al match siguiente.
- `generateRoundRobin(participants, rounds)`: algoritmo de círculo, una o dos vueltas.
- `generateSwissRound(standings, previousPairings)`: empareja por récord sin repetir rivales; asigna bye.
- `buildGroups(participants, groupSize)`: distribución serpenteante por siembra.
- `computeStandings(matches, tiebreakers)`: victorias, diferencia de puntos, enfrentamiento directo, Buchholz.
- `advanceStage(stage, standings)`: obtiene los clasificados y crea la etapa siguiente, incluido un bracket que inicia en 16vos.
- `applyBattle(match, battle, config)`: replica en TypeScript la lógica de la función SQL record_battle (puntos, sets y fin del match) para pruebas y vista previa; la fuente de verdad es la función SQL, que corre en una sola transacción.

Cada función se prueba con casos de 2 a 64 participantes, número impar, byes y descalificaciones a mitad de torneo.

10. Plan por etapas de desarrollo

1. **Fundamentos:** repositorio, CLAUDE.md, proyecto Supabase, migración inicial, autenticación y roles, despliegue en Vercel.

2. **Motor de torneos:** funciones de la sección 9 con pruebas unitarias antes de cualquier interfaz.
3. **CRUD de torneos y participantes:** creación, edición, inscripción manual y siembra.
4. **Etapas y brackets:** generación, vista previa y visualización del bracket y de los grupos.
5. **Arbitraje:** panel móvil, batallas, tarjetas, corrección de resultados y bitácora.
6. **Tiempo real y vista pública:** enlaces públicos, Realtime y exportación.
7. **Pruebas integrales y endurecimiento:** E2E con Playwright, revisión de RLS, rendimiento y copias de seguridad.
8. **Lanzamiento y mantenimiento:** documentación, monitoreo y backlog de la versión 2 (ELO, estadísticas).

11. Decisiones tomadas

- Las cuentas de organizador y árbitro se crean por invitación del admin; el registro público queda cerrado.
- Los desempates son configurables por etapa.
- Cada 3 faltas de un participante en el match dan 1 punto al rival (ambos valores configurables por torneo).
- La eliminación directa se juega al mejor de 3 sets por defecto; grupos y suizo a un solo set. Configurable por etapa.
- Tras los grupos, pasan los primeros N de cada grupo según la tabla (N configurable por etapa y por grupo).
- La Fase 2 (rol blader con autoinscripción, decks y piezas, pantalla de estadio y ranking ELO) se decide cuando el MVP esté en producción.